

Padrões de Projeto — SigBah!

Disciplina: INF01127 — Engenharia de Software N

Professora: Dra. Lucinéia Heloisa Thom

Sistema: SigBah! — Sistema de Gestão de Bancas Acadêmicas (PPGFis / PPGenFis — UFRGS)

Integrantes:

- Lucas Nogueira
- Gabriel Stoffel
- João Vitor
- Fidel Luz
- Ricardo Carvalho

1. Visão Geral do Sistema

O SigBah! é um sistema web para gestão de bancas acadêmicas dos programas de pós-graduação em Física e Ensino de Física da UFRGS. O sistema permite:

- Submissão de pedidos de banca por servidores/orientadores
- Aprovação ou rejeição de pedidos pelo coordenador (via e-mail com link)
- Geração automática de documentos (Ata, Cartas de Convite, Pareceres, Cartaz, Relatoria)
- Administração de bancas com versionamento de edições

Stack tecnológica:

- Backend: Python / FastAPI / MongoDB (pymongo)
- Frontend: React / TypeScript / react-hook-form / Zod / react-router-dom
- Infraestrutura: Docker (MongoDB + Mailpit + Postfix)

2. Padrões de Projeto Implementados

2.1 DAO (Objeto de Acesso a Dados)

Classificação: Padrão de Acesso a Dados

Objetivo: Separar o modelo de dados da aplicação da tecnologia de persistência, promovendo isolamento e flexibilidade — principalmente quando se deseja trocar o SGBD.

Justificativa no contexto do SigBah!: O sistema utiliza MongoDB como banco de dados, mas a camada de aplicação (`BancaService` , `AdminBancaService`) não conhece detalhes do MongoDB. A interface abstrata `BancaRepository` define as operações de persistência (incluir, buscar, atualizar), e a implementação concreta `MongoBancaRepository` realiza essas operações usando pymongo. Isso permite trocar o banco de dados sem alterar a lógica de negócio.

Implementação no código:

```
# app/domain/banca_repository.py – Interface abstrata do DAO
class BancaRepository(ABC):
    @abstractmethod
    def save(self, req: BancaRequest) -> Result[str, PersistenceError]: ...
```

```

@abstractmethod
def find_by_token(self, token: str) -> Result[BancaRecord, BancaNotFoundError | PersistenceError]: ...

@abstractmethod
def update_decision(self, token: str, status: BancaStatus, reason: str | None = None) -> Result[...]: ...

@abstractmethod
def list(self, filters: BancaListFilters) -> Result[list[BancaListItem], PersistenceError]: ...

@abstractmethod
def append_version(self, token: str, req: BancaRequest) -> Result[int, ...]: ...

```

```

# app/infrastructure/mongo_banca_repository.py – Implementação concreta do DAO
class MongoBancaRepository(BancaRepository):
    def save(self, req: BancaRequest) -> Result[str, PersistenceError]:
        token = str(uuid.uuid4())
        doc = { "versions": [...], "decision_token": token, "status": "pending", ... }

        get_db()["bancas"].insert_one(doc)
        return Ok(token)

    def find_by_token(self, token: str) -> Result[BancaRecord, ...]:
        doc = get_db()["bancas"].find_one({"decision_token": token})
        ...

```

A coleção `bancas` no MongoDB funciona como o DAO do sistema — encapsulando operações de incluir, remover, pesquisar e atualizar bancas.

2.2 Facade (Fachada)

Classificação: Padrão Estrutural

Objetivo: Prover uma interface simplificada para um subsistema complexo, reduzindo o acoplamento entre os clientes e os componentes internos.

Justificativa no contexto do SigBah!: A aprovação de uma banca envolve múltiplas operações: buscar no banco, gerar documentos PDF, enviar e-mail com anexo, e atualizar o status. O `BancaService` encapsula toda essa orquestração em um único método `approve()`, oferecendo uma interface simples para a camada de API.

Implementação no código:

```

# app/application/banca_service.py
class BancaService:
    def __init__(self, repo: BancaRepository) -> None:
        self._repo = repo

    def approve(self, token: str) -> Result[BancaDecisionResponse, ...]:

```

```

# 1. Busca a banca no repositório
match self._repo.find_by_token(token):
    case Err() as err: return err
    case ok: record = ok.value

# 2. Gera documentos PDF
match document_service.generate_documents(req):
    case Err() as err: return err
    case ok: zip_bytes, zip_name = ok.value

# 3. Envia e-mail com documentos à secretaria
match email_service.send_documents_email(SECRETARY_EMAIL, ...):
    case Err() as err: return err

# 4. Atualiza status no banco
match self._repo.update_decision(token, "approved"):
    case Err() as err: return err

return Ok(BancaDecisionResponse(...))

```

O router da API chama apenas `banca_service.approve(token)` — não precisa conhecer os detalhes de geração de documentos, envio de e-mail ou persistência. O `BancaService` é a fachada que simplifica o acesso ao subsistema complexo.

2.3 Singleton

Classificação: Padrão de Criação

Objetivo: Assegurar que uma classe tenha uma única instância e prover um ponto de acesso global a ela.

Justificativa no contexto do SigBah!: A conexão com o MongoDB deve ser única durante toda a vida da aplicação para evitar múltiplas conexões desnecessárias e garantir reuso do pool de conexões.

Implementação no código:

```

# app/infrastructure/database.py
_client: MongoClient | None = None

def get_db() -> Database:
    global _client
    if _client is None:
        _client = MongoClient(
            MONGO_URI,
            username=MONGO_USERNAME or None,
            password=MONGO_PASSWORD or None,
        )
    return _client[MONGO_DB]

```

A variável global `_client` é inicializada apenas uma vez (lazy initialization). Todas as chamadas subsequentes a `get_db()` reutilizam a mesma instância do `MongoClient`, garantindo que exista um único ponto de acesso ao banco de dados.

2.4 Strategy (Estratégia)

Classificação: Padrão Comportamental

Objetivo: Definir uma família de algoritmos, encapsular cada um deles e torná-los intercambiáveis.

Justificativa no contexto do SigBah!: O sistema gera diferentes tipos de documentos PDF (Ata, Carta de Convite, Parecer, Cartaz, Relatoria de Avaliação). Cada tipo de documento é uma "estratégia" de geração, selecionada dinamicamente com base no manifesto de arquivos solicitados.

Implementação no código:

```
# app/application/document_service.py
def generate_files(req: BancaRequest, ids: list[str]) -> Result[...]:
    needed_kinds = {entry.kind for entry in requested}

    # Cada "kind" dispara uma estratégia de geração diferente
    if "ata" in needed_kinds:
        banca.criaAta(save=True)
    if "cartaz" in needed_kinds:
        banca.criaCartaz(save=True)
    if "relatoria_avaliacao" in needed_kinds:
        banca.criaRelatoriaAvaliacao(save=True)
    if "carta_convite" in needed_kinds:
        banca.criaCartaConvite(save=True)
    if "parecer" in needed_kinds:
        banca.criaParecer(save=True)
```

O `file_manifest()` define quais documentos existem para cada banca (variando por tipo — mestrado, qualificação, doutorado), e `generate_files()` executa apenas as estratégias de geração necessárias.

2.5 Composite (Composição)

Classificação: Padrão Estrutural

Objetivo: Compor objetos em estruturas de árvore para representar hierarquias parte-todo, permitindo que clientes tratem objetos individuais e composições de forma uniforme.

Justificativa no contexto do SigBah!: O frontend React utiliza composição de componentes para construir formulários complexos a partir de peças reutilizáveis simples. Componentes folha e componentes compostos são tratados uniformemente pela árvore de renderização.

Implementação no código:

```
// Componente folha: Field (wrapper com label)
export default function Field({ label, required, children }: FieldProps) {
    return (
        <label>
            <span>{label}{required && <span>*</span></span>}</span>
            {children}
        </label>
    )
}
```

```

    );
}

// Componente folha: TextInput
export default function TextInput(props: TextInputProps) {
    return <input className={baseClassName} {...props} />;
}

// Componente composto: MemberField (compõe Field + TextInput + SelectInput)
export default function MemberField({ label, name, ... }: MemberFieldProps) {
    return (
        <section>
            <Field label="Tratamento" required>
                <SelectInput {...register(`${name}.gender`)} />
            </Field>
            <Field label="Nome" required>
                <TextInput {...register(`${name}.name`)} />
            </Field>
            ...
        </section>
    );
}

// Composição de nível superior:
// BancaForm → BancaGeneralSection + BancaCompositionSection → MemberField → Field +
// TextInput

```

Componentes simples (`TextInput` , `SelectInput` , `Field`) são compostos em componentes mais complexos (`MemberField`), que por sua vez compõem seções (`BancaCompositionSection`), que compõem o formulário completo (`BancaForm`). A árvore inteira é tratada uniformemente pelo React.

3. Padrões de Projeto Sugeridos

3.1 State (Estado)

Classificação: Padrão Comportamental

Objetivo: Permitir a um objeto alterar seu comportamento quando seu estado interno mudar, como se o objeto mudasse de classe.

Justificativa no contexto do SigBah!: Uma banca possui um ciclo de vida com estados bem definidos (`pending` → `approved` | `rejected`). Atualmente, as regras de transição estão espalhadas em condicionais (`if record.status !== "pending"` , `if current_status !== "approved"`). O padrão State encapsularia cada estado em uma classe com suas operações permitidas.

Benefício: Elimina condicionais repetidos sobre `status` nos serviços, centraliza as regras de transição, e facilita a adição de novos estados (ex.: `"em_revisão"` , `"cancelada"`).

3.2 Observer (Observador)

Classificação: Padrão Comportamental

Objetivo: Definir uma dependência um-para-muitos entre objetos, de modo que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

Justificativa no contexto do SigBah!: Quando uma banca é aprovada, múltiplas ações ocorrem: envio de documentos à secretaria, notificação ao orientador, geração de PDFs. Atualmente essas ações estão acopladas sequencialmente no método `approve()`. Com Observer, cada ação seria um observador independente registrado no evento "banca aprovada".

Benefício: Desacopla a lógica de decisão das ações consequentes. Novos observadores (ex.: log de auditoria, webhook para sistema externo) podem ser adicionados sem modificar o `BancaService`.

3.3 Factory Method (Método Fábrica)

Classificação: Padrão de Criação

Objetivo: Definir uma interface para criar objetos, mas deixar as subclasses decidirem qual classe instanciar.

Justificativa no contexto do SigBah!: O sistema gera diferentes tipos de documentos PDF. Atualmente, a seleção é feita por condicionais (`if "ata" in needed_kinds`). Um Factory Method encapsularia a criação de cada gerador de documento, tornando o sistema aberto para extensão e fechado para modificação.

Benefício: Cada tipo de documento tem sua própria classe geradora. Adicionar um novo tipo de documento (ex.: "declaração de participação") requer apenas criar uma nova classe e registrá-la, sem alterar a lógica existente.

3.4 Template Method (Método Template)

Classificação: Padrão Comportamental

Objetivo: Definir o esqueleto de um algoritmo em uma operação, delegando alguns passos para subclasses.

Justificativa no contexto do SigBah!: O sistema envia diferentes tipos de e-mail (petição, rejeição, documentos). Todos seguem a mesma estrutura: criar mensagem MIME, definir headers, anexar corpo HTML, opcionalmente anexar arquivo, e enviar via SMTP. Atualmente há duplicação entre `send_petition_email`, `send_rejection_email` e `send_documents_email`.

Benefício: Elimina duplicação de código entre os diferentes tipos de e-mail. O algoritmo de envio é definido uma vez no template; cada tipo de e-mail apenas preenche os passos variáveis.

4. Conclusão

O SigBah! já emprega padrões de projeto que promovem baixo acoplamento, alta coesão e facilidade de manutenção. O DAO garante independência da tecnologia de persistência; a Fachada simplifica a orquestração de operações complexas; o Singleton evita desperdício de recursos na conexão com o banco; o Strategy permite geração seletiva de documentos; e o Composite estrutura a interface de forma modular e reutilizável.

Os padrões sugeridos (State, Observer, Factory Method e Template Method) representam evoluções naturais que poderiam ser adotadas conforme o sistema cresce em complexidade — por exemplo, ao adicionar novos estados ao ciclo de vida da banca, novos tipos de notificação, ou novos formatos de documento. Sua implementação, entretanto, depende de questões como tempo disponível antes da

entrega final e por decidir se o custo de implementação realmente vale a pena dado o contexto e objetivo do projeto.